EURECOM
Sophia Antipolis

Cloudspaces Storage (WP3)

Bruxelles, December 11, 2013



cloudspaces

WP3: Cloudspaces Storage

- **Task 3.1 Personal Storage [EUR, URV, TST, EOS]**
  - *aggregate the different **heterogeneous storage resources***
  - ***optimized user-percieved latency** for data replication/synchronization*
  - *new data management strategies addressing **reconfigurability, heterogeneity, consistency, availability, performance and cost.***
- **Task 3.2 Cloud Storage [TST, EUR, URV]**
  - *modify an open source Cloud storage solution (OpenStack Swift) according to Task 3.1*
  - *customizable storage offerings with different redundancy requirements.*
  - ***novel Cloud-provided seed swarming schemes** to address scalability of Personal clouds*
- **Task 3.3 Proximity-Aware Sync and Share**
  - M12-M28

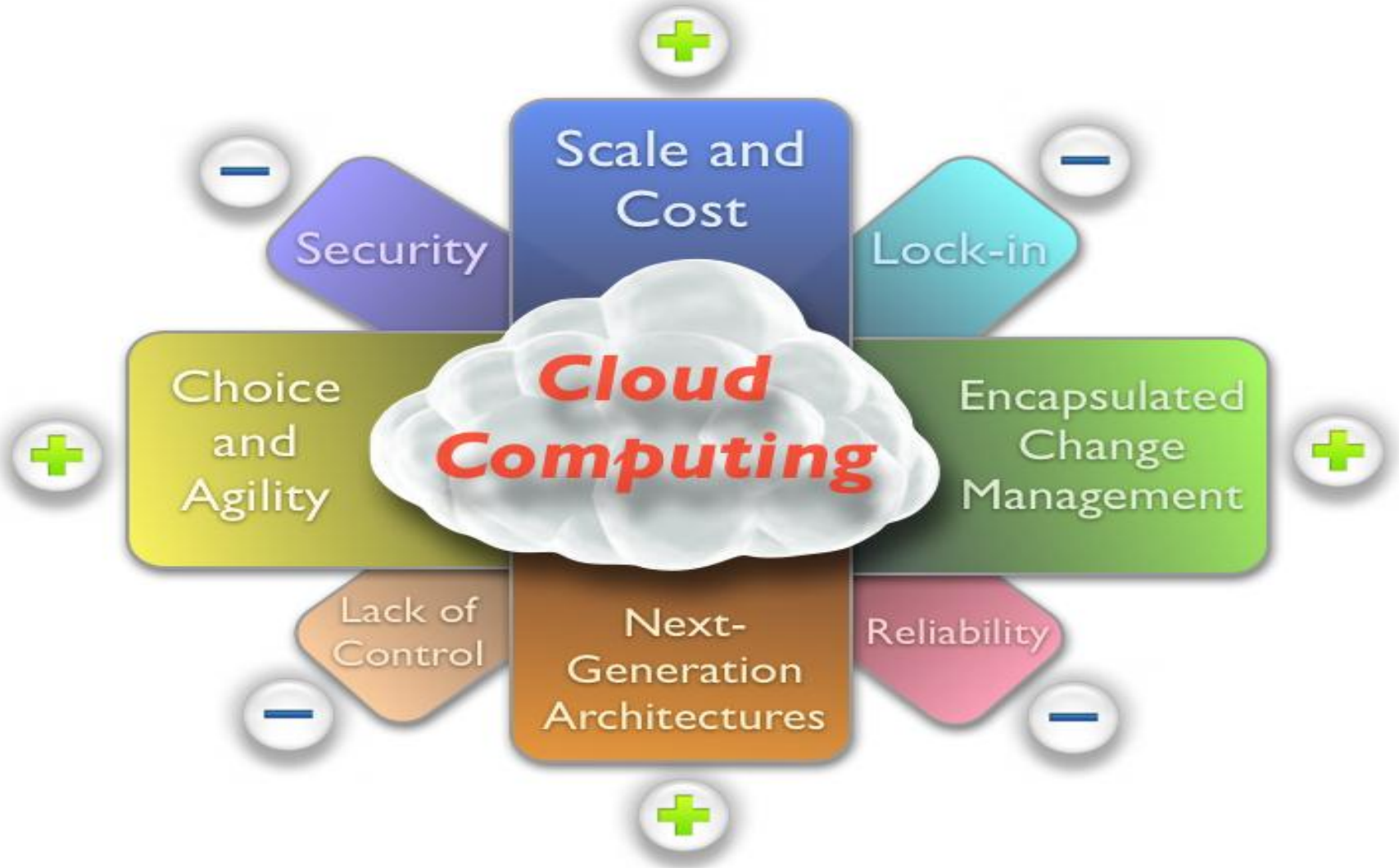
WP3: Put differently

- **Hybrid cloud storage [EUR]**
  - Leverage multiple, heterogeneous cloud resources
  - Improve user percieved latency, reconfigurability, heterogeneity, consistency, availability, performance and cost

- **Adaptive edge platform [URV, TST]**
  - Boost efficiency and scalability of personal clouds using multiple, heterogeneous personal cloud clients
  - BitTorrent among swarms of heterogenous personal cloud
  - prototype has been massively deployed by URV and TISSAT.

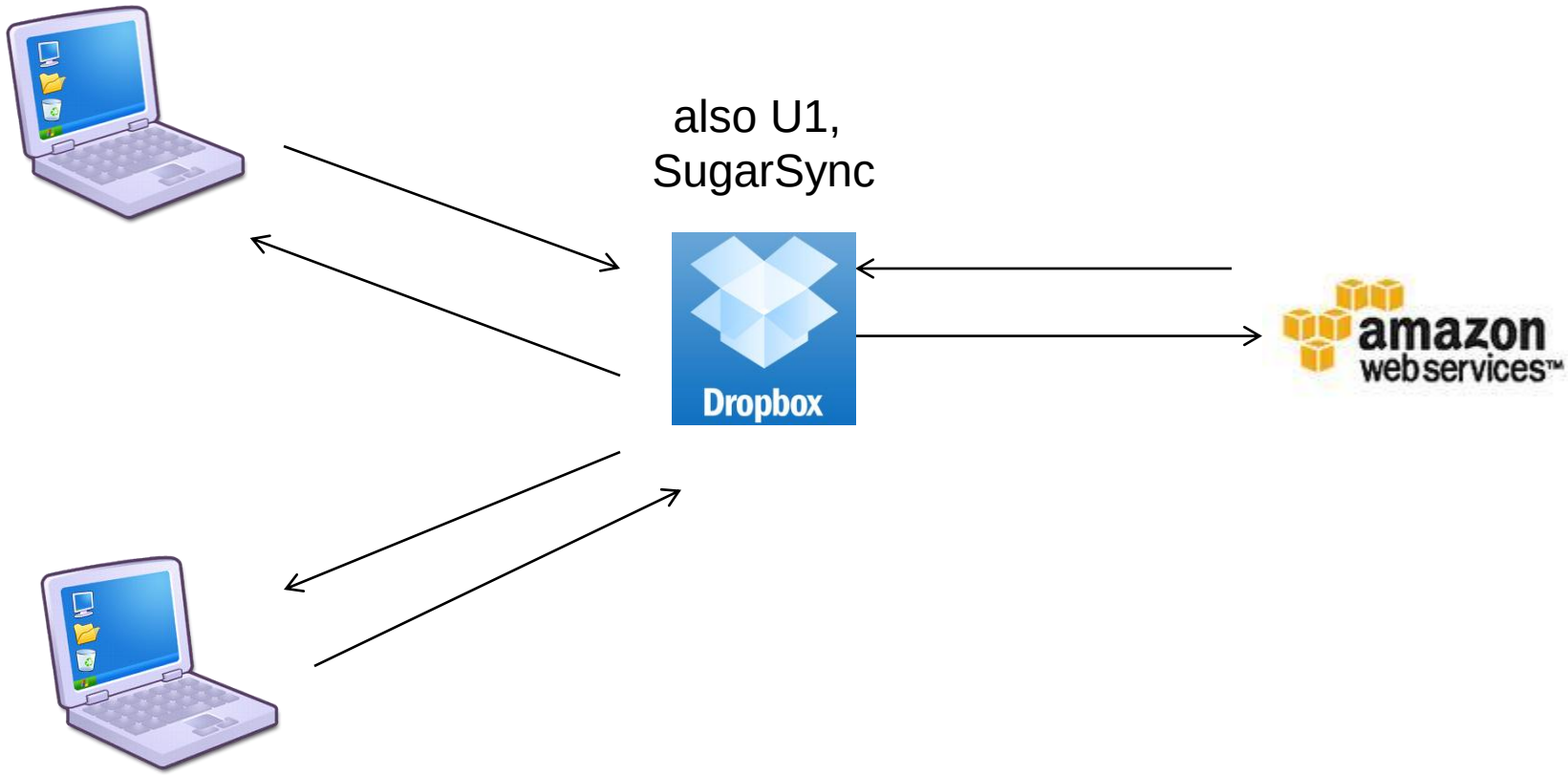
- **D 3.1.: Guidelines for heterogeneous personal clouds**

Pros and Cons



From http://blogs.zdnet.com/Hinchcliffe

Current Personal Clouds (simplified view)



Solution?

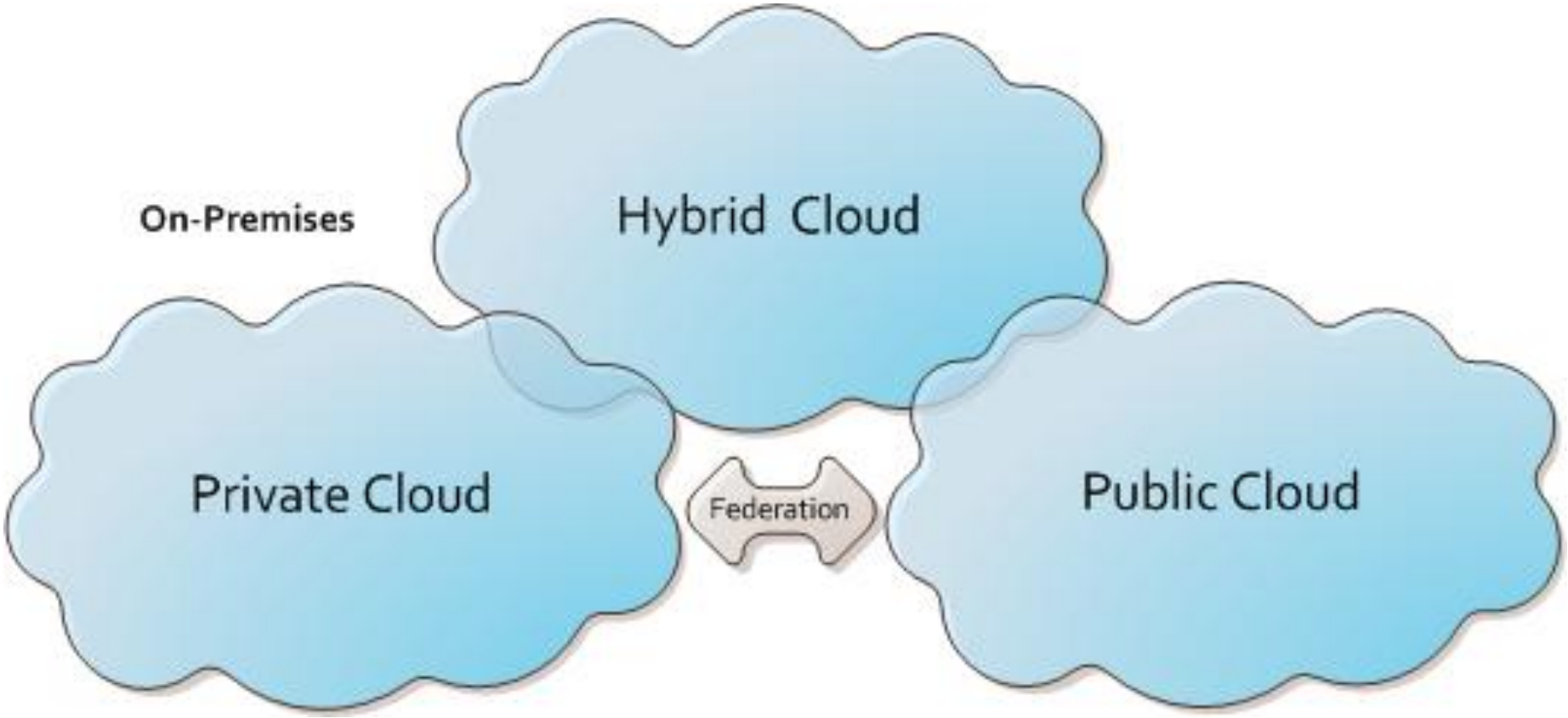
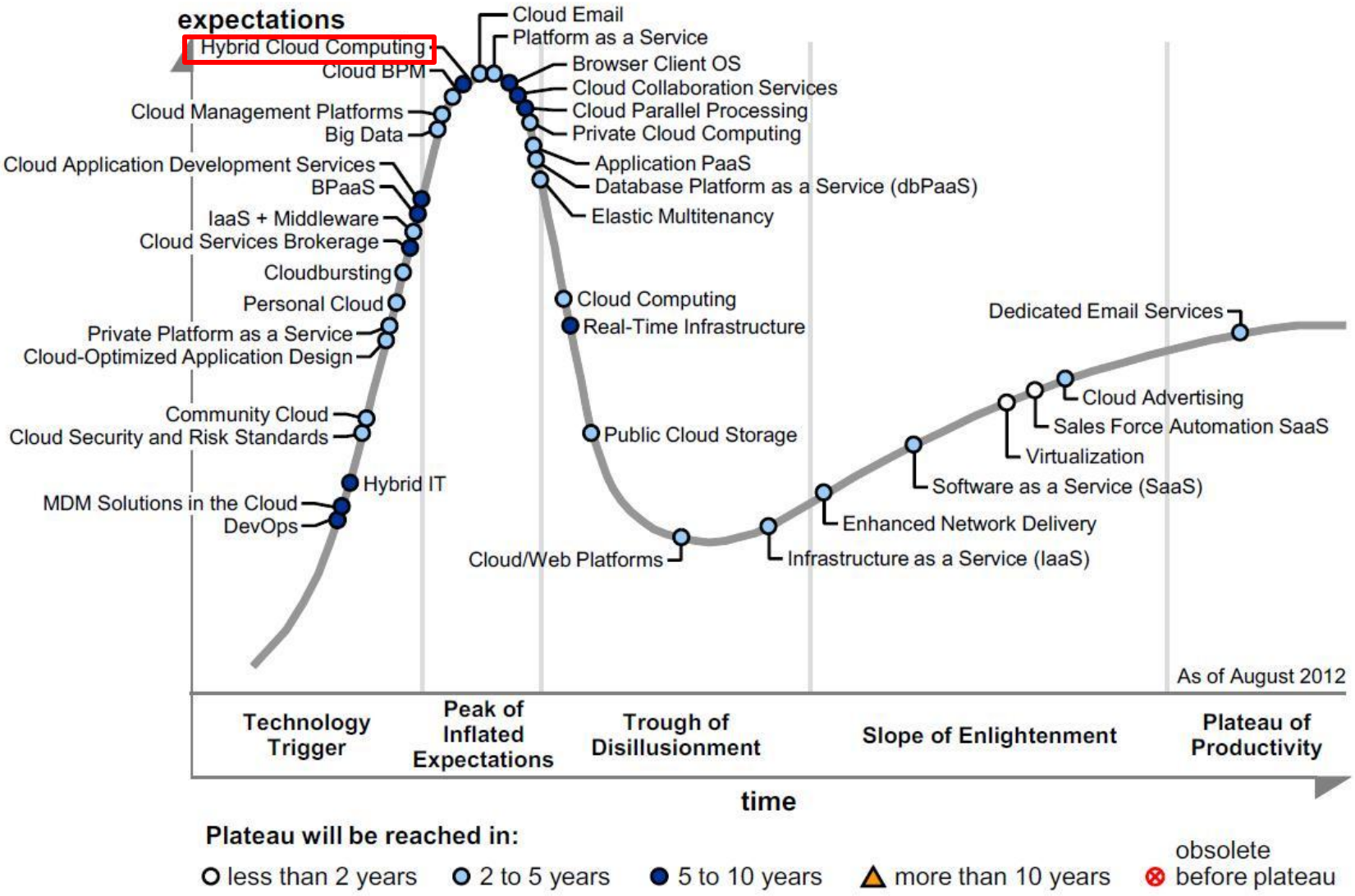

On-Premises
Hybrid Cloud
Private Cloud
Federation
Public Cloud

Figure 1. Hype Cycle for Cloud Computing, 2012



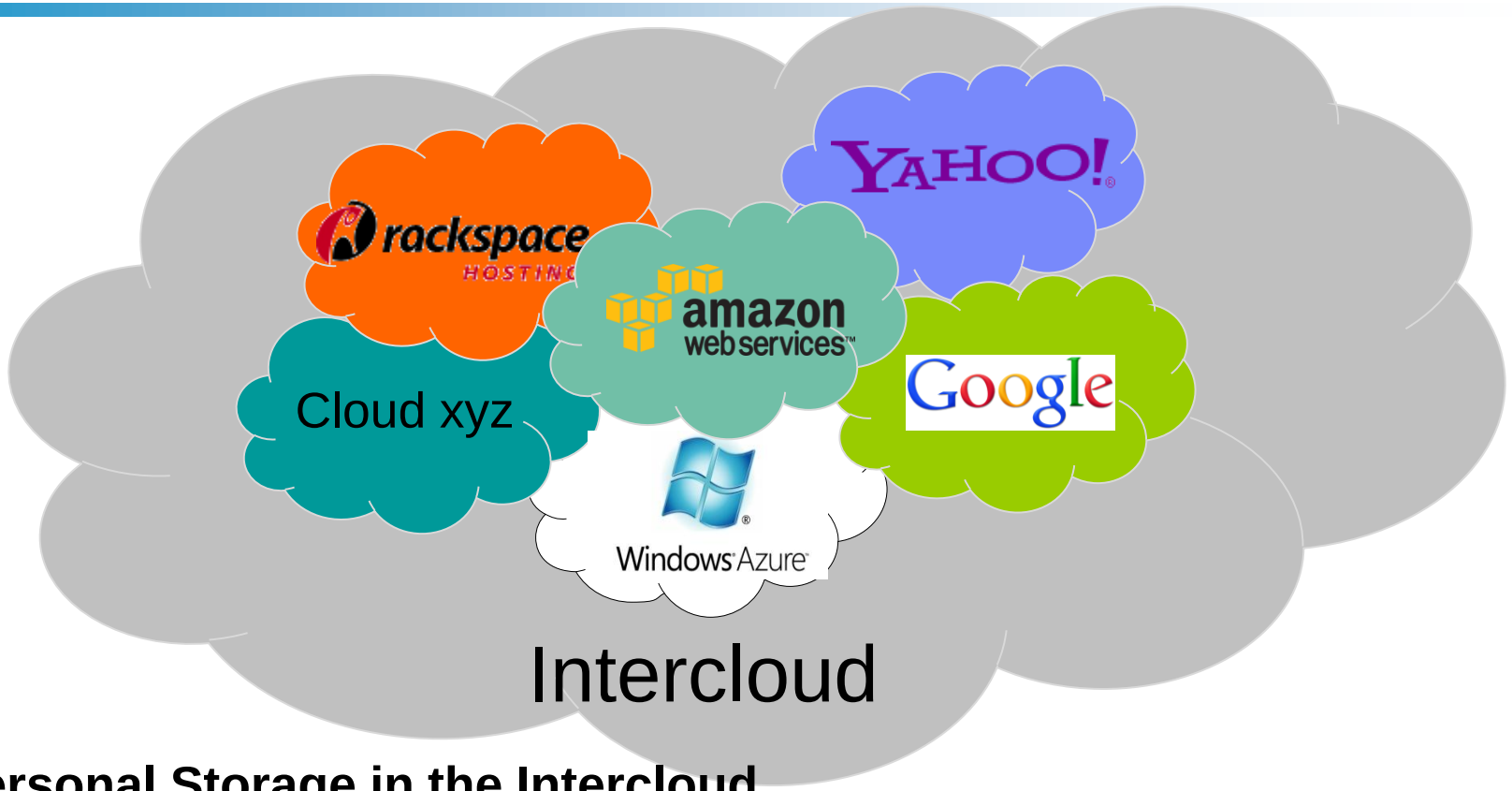
Source: Gartner (August 2012)

Hybrid Cloud

+ Security
+ Control

- Reliability
- Lock-in
- Proprietary Solutions

Reality: The Intercloud



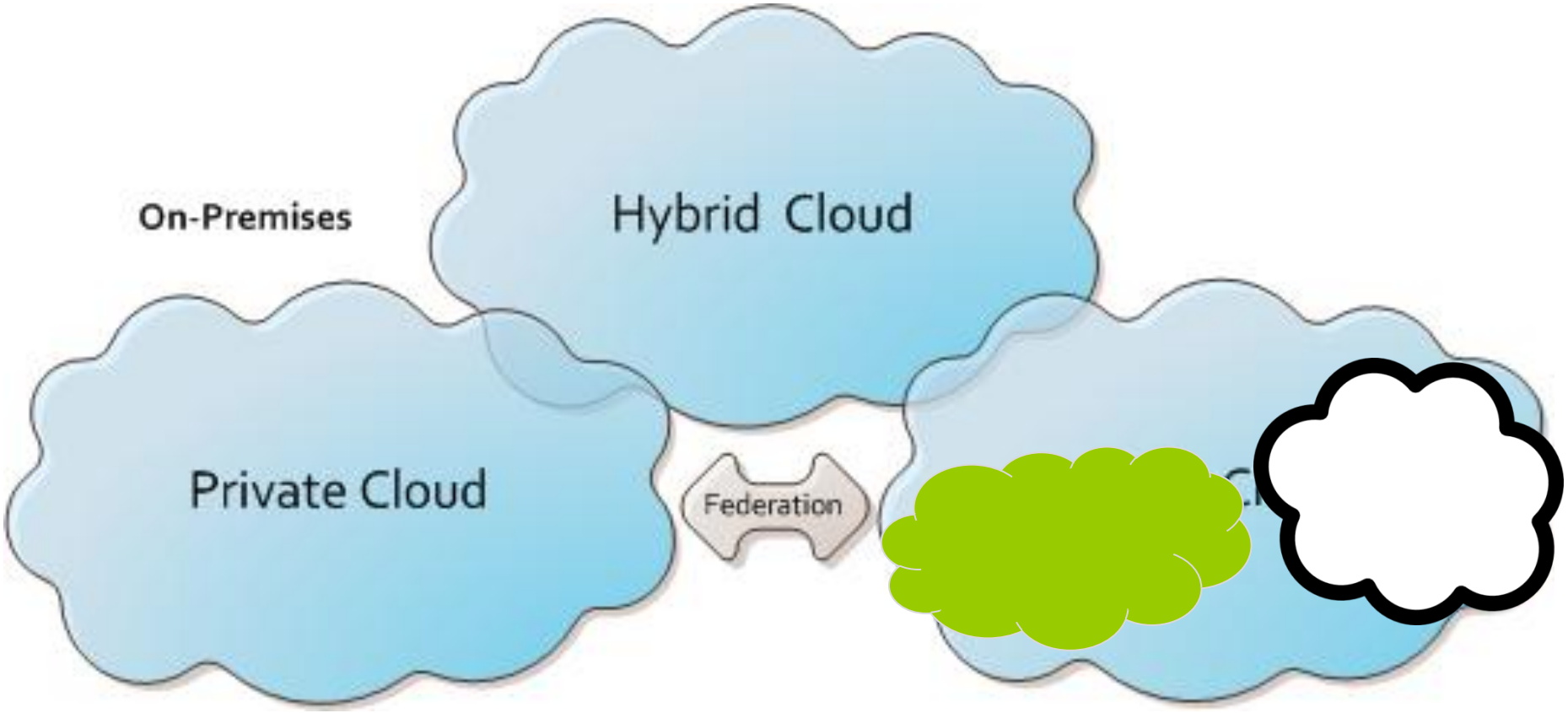
- **Personal Storage in the Intercloud**
  - **Distributes trust** across several providers
  - **Addresses lock-in**
  - **Reliability:**
    - **Heterogeneity** among clouds ensures genuine fault independence

Issues?

- **Security (key management)?**

- **Metadata management?**

Our approach: Robust Hybrid Cloud Storage



Intercloud

What is HCS?

- **HCS is a key-value storage system for hybrid cloud**
  - Multi-writer multi-reader, support for heavy concurrency
  - Strong consistency (linearizability)
  - Robust: no single point of failure, tolerates malicious public clouds
- **Exports the key-value store (KVS) API**
  - as Amazon S3, OpenStack Swift
- **KVS API**
  - put(key, value)
  - get(key)
  - list() (returns a list of keys within a container)
  - delete(key)
  - …. (but the above 4 are the main ones)

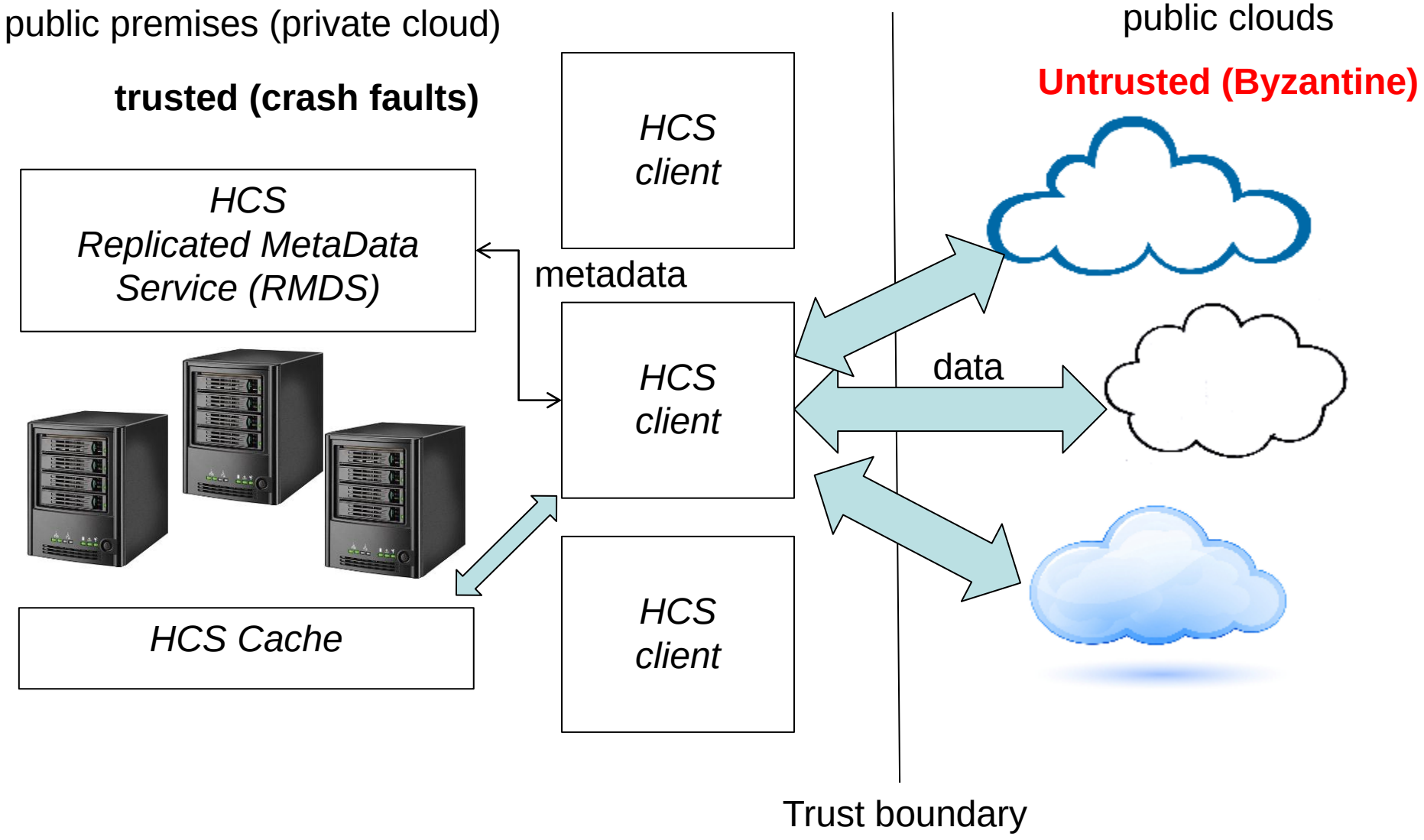
HCS: the key idea



Keep metadata on private premises

(i.e., separately from data outsourced to public clouds)

HCS model/architecture



HCS: Novel features

- **Separating metadata from data**
  - yields surprising advantages!

- **Consistency hardening** 

  - Leverage RMDS to mask inconsistent clouds

Strong metadata consistency

extends to strong data consistency

despite weak (eventual) cloud consistency!

- **Boosting reliability** 

  - Can tolerate *f* untrusted (Byzantine) clouds using only 2f+1 clouds!
  - **Wipes out the gap between crash and Byzantine tolerance**

HCS is very efficient!

- **In common case** **[no faults, consistent clouds]**
  - Writes replicate data only to f+1 clouds
  - Reads read data from a single cloud

- **Reads**
  - never involve more than f+1 clouds
  - even in the worst case [(malicious) faults, inconsistencies, outages,…]

- **Writes**
  - may need 2f+1 clouds in the worst case for availability

Metadata service

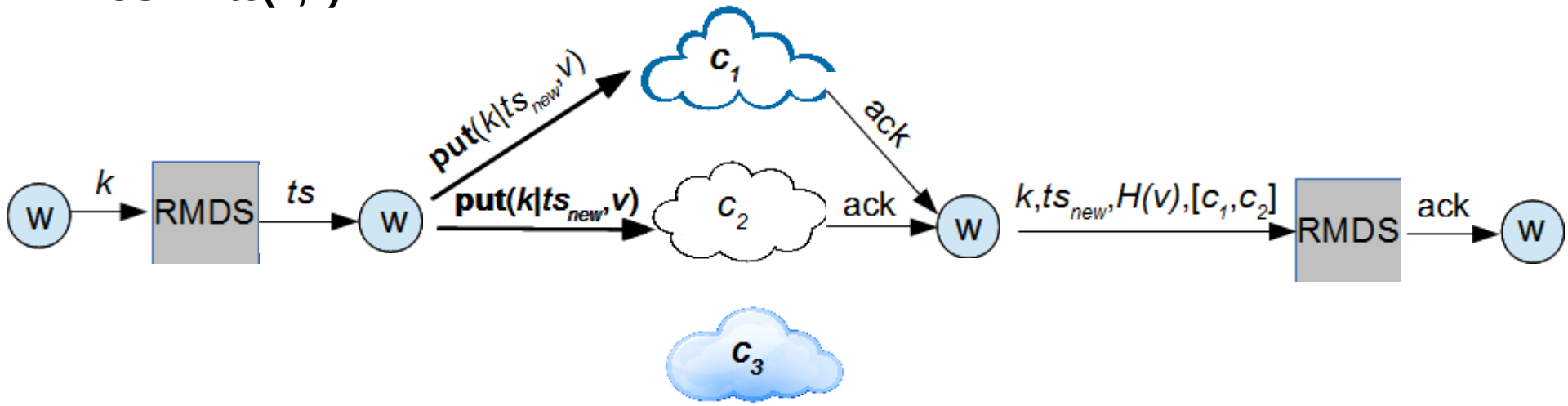
- **HCS metadata service (RMDS) stores, per object**
  - Timestamp
    - ☞ version number of the object
  - Directory info
    - ☞ Pointers to f+1 clouds that store the latest copy of the object
  - Hash
  - Object size
  - [Encryption key]
    - ☞ optional, for data confidentiality

- **Metadata is very lightweight – critical for scalability**
  - ~40 bytes metadata

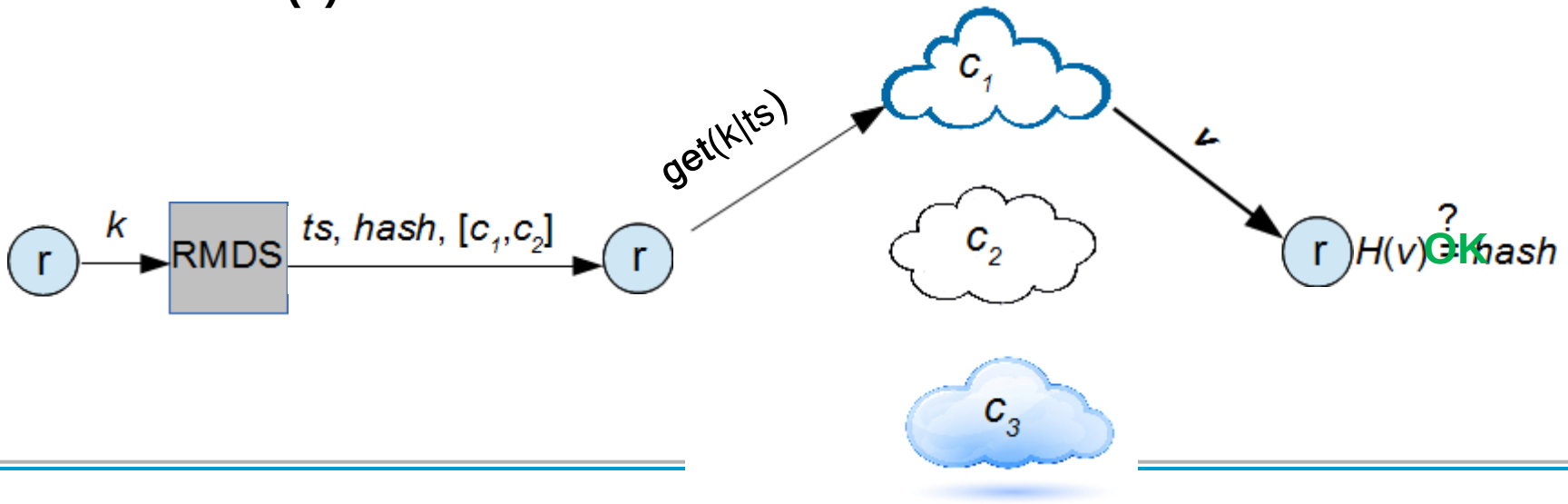
How does HCS work (common case)?

HCS.write(k,v)

f=1



HCS.read(k)



HCS implementation

- **The key: implementing RMDS and client-RMDS interaction**
  - Robustly!
  - Without any single point of failure

- **Implementation decision**
  - Use Zookeeper for RMDS
  - …or HBase (and some other NoSQLs)…
  - …or SQL-based replicated DBMS…

- **The main condition**
  - HCS RMDS needs conditional updates

HCS Prototype

- **Java**
- **Wrapper around ZooKeeper client library**
- **Interfaces to clouds wrapping their native clients**

Deployment

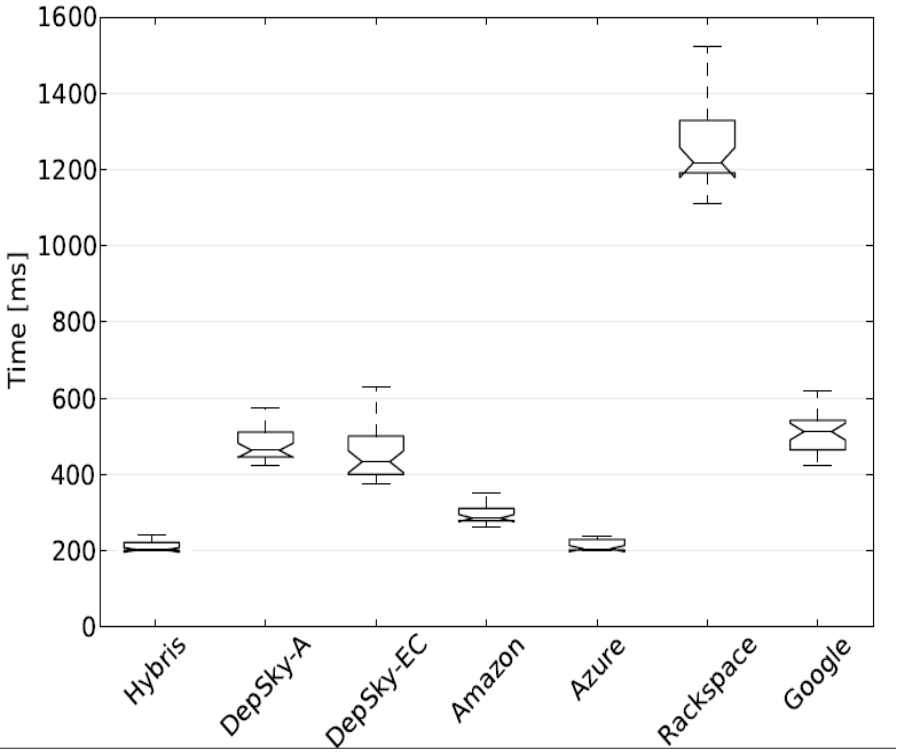


- **Private cloud**
  - FP7 BIGFOOT OpenStack cluster @EURECOM
  - ZooKeeper servers and clients running on Openstack VMs

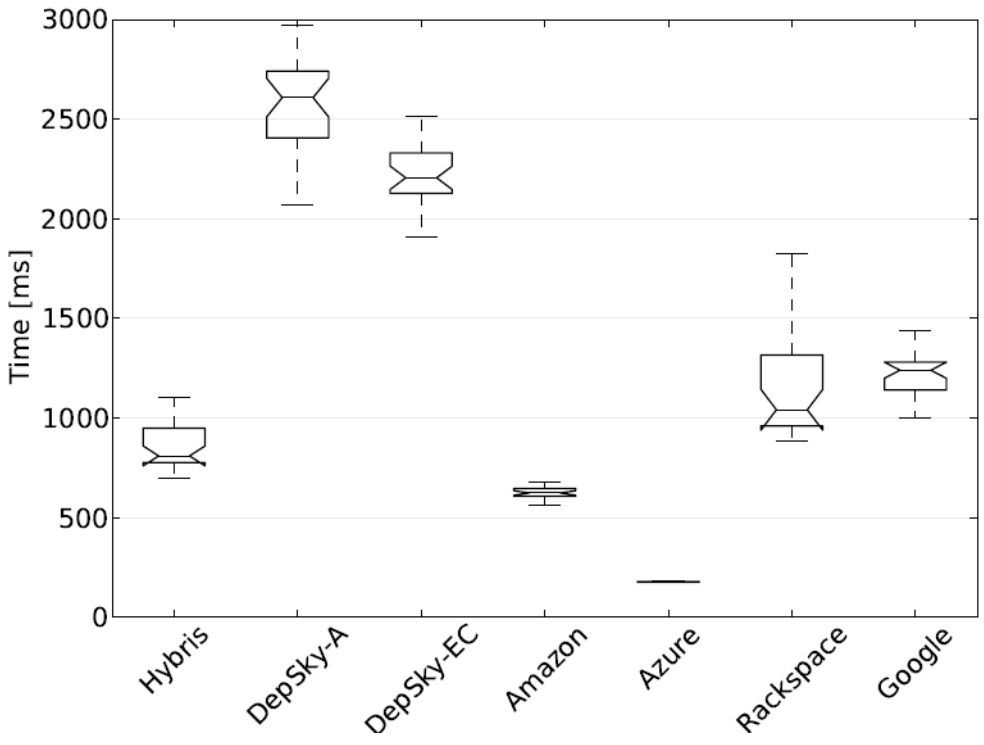
- **Public clouds**
  - Google, Amazon S3, Azure, Rackspace

Performance (latency, 100kB)

reads



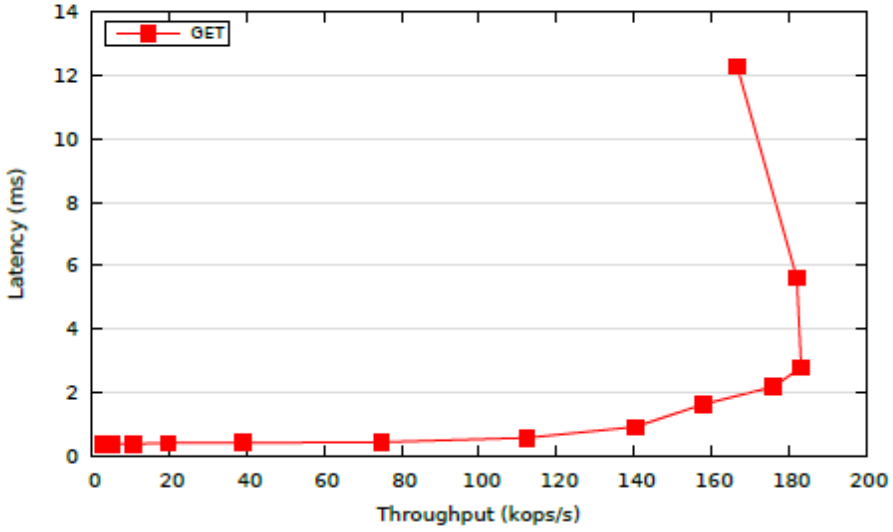
writes

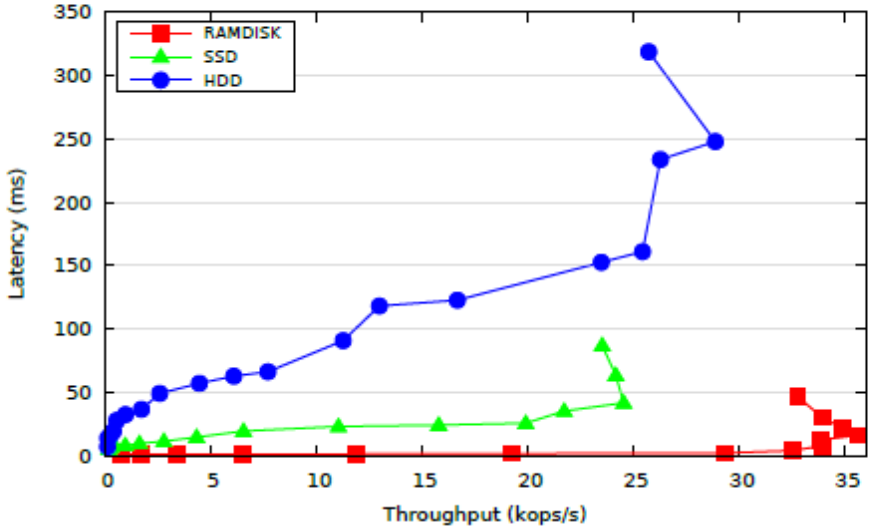


EURECOM
Sophia Antipolis

HCS Scalability

- **Short-circuiting clouds (to stress RMDS) we obtain**
  - With 3-node Zookeeper deployment





EURECOM
Sophia Antipolis

Next Steps (coarse grained)



- **Basic Integration with URV Stacksync front-end**
  - Basic integration with HCS and Stacksync
  - Proof-of-concept of the “Dropbox for SMEs”

- **Support for erasure coding in HCS**

WP3

- **Hybrid cloud storage [EUR]**
  - Leverage heterogeneous cloud resources
  - Optimize for user perceived latency
  - Address reconfigurability, heterogeneity, consistency, availability, performance and cost

- **Adaptive edge platform**
  - Boost efficiency and scalability of personal clouds using swarms
  - BitTorrent among swarms of heterogenous personal clouds
  - prototype has been massively deployed by two Cloudspaces partners: URV and TISSAT.

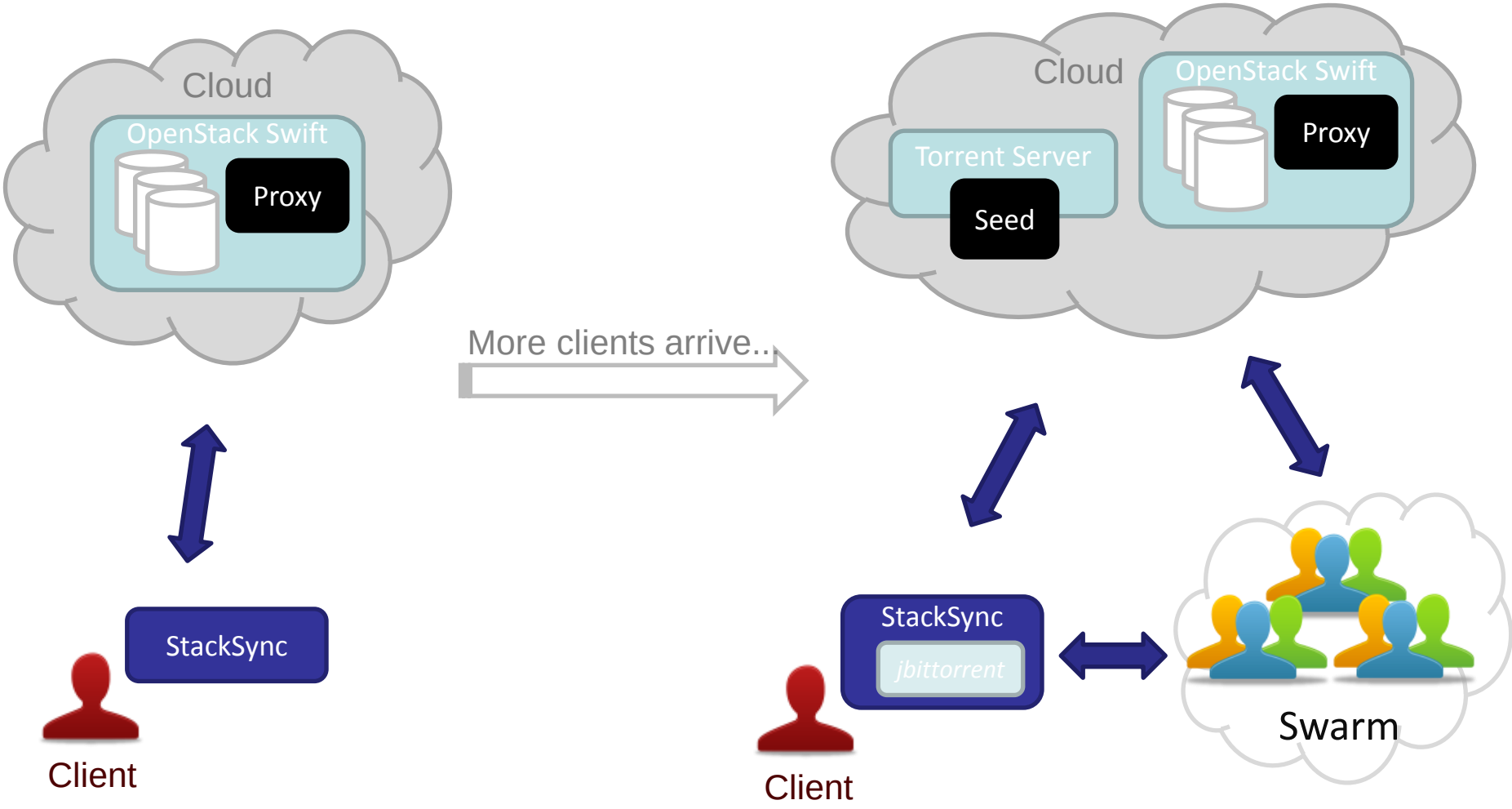
- **D 3.1.: Guidelines for heterogeneous personal clouds**

Adaptive Edge Platform: Concept

- **Idea**
  - **Adaptive threshold** for **transparent switch** from classic *HTTP* to *BitTorrent* upon detection of a certain critical **mass** on a specific content

- **Gain**
  - Taking advantage of the peer-to-peer technologies to enhance content distribution
  - **Offloading the server** from doing all the serving thanks to the peers' contribution

System Architecture



(a) Before the switch to BitTorrent

(b) After the switch to BitTorrent

Switching Metrics

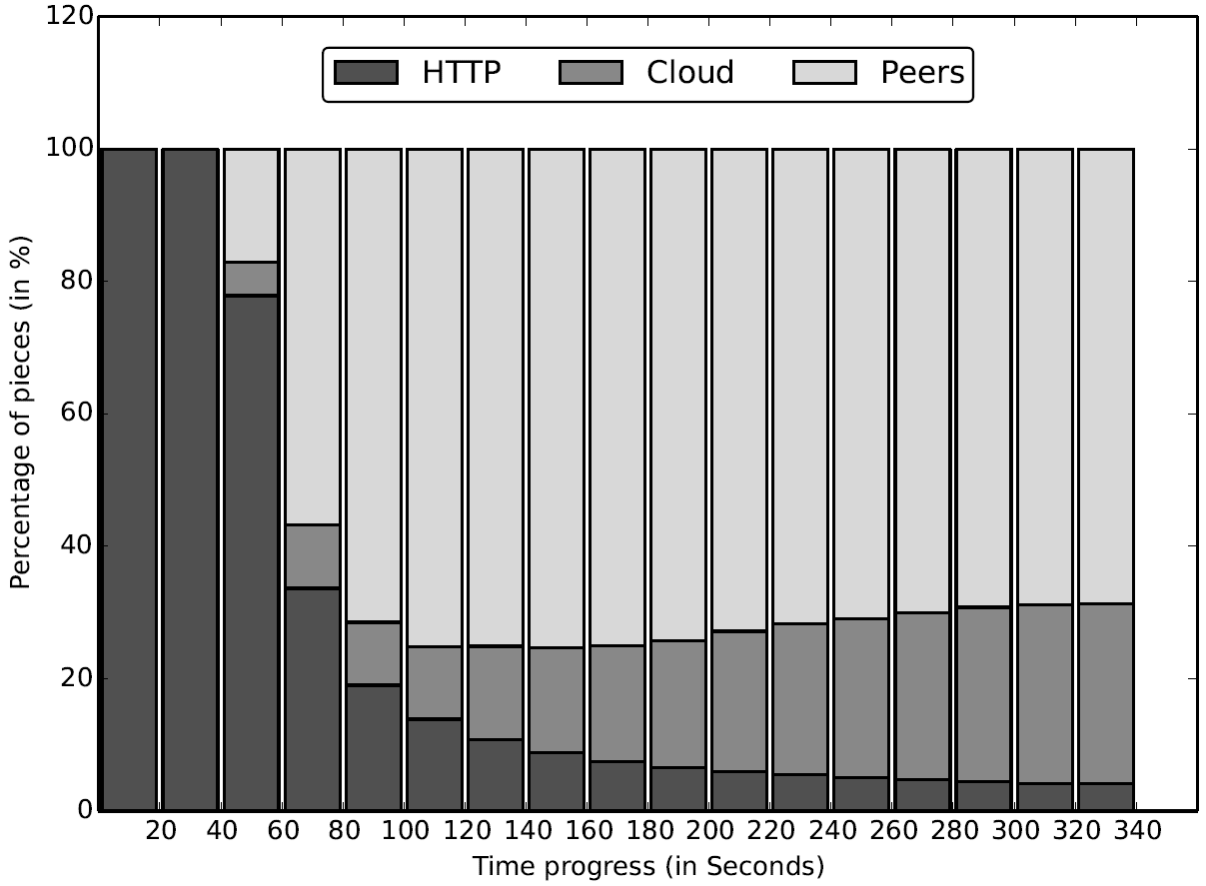
- **Parameters to be considered:**
  - The size of the shared file (th_{size})
  - The number of the clients sharing the file ($th_{clients}$)
  - Their aggregated bandwidth

- **Switching Process:**
  - Monitoring popular shared files

    ($number_{clients} > th_{clients}$ && $file_{size} > th_{clients}$)
  - When the aggregated download bandwidth of the swarm is greater than the seed's upload bandwidth, we switch to BitTorrent
    - The cloud is no longer able to saturate the clients' download bandwidth

Results

Offload can reach 60% in most common cases



EURECOM
Sophia Antipolis

Thank you!